

# Machine Learning

## Lecture 9: SVM and Kernels

---

Prof. Dr. Stefan Leutenegger

Smart Robotics Lab  
Technical University of Munich

Winter term 2024/2025

# Roadmap

1. Support Vector Machines (SVM)
2. Soft Margin Support Vector Machines
3. Kernels

## Section 1

# Support Vector Machines (SVM)

# Linear classifier

A linear classifier assigns all  $x$  with

$$\mathbf{w}^T \mathbf{x} + b > 0$$

to class **blue** and all  $x$  with

$$\mathbf{w}^T \mathbf{x} + b < 0$$

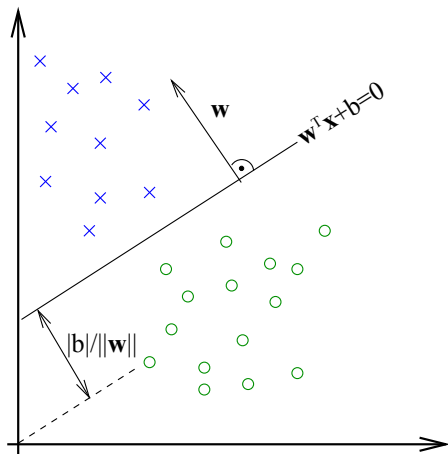
to class **green**.

Thus the class of  $x$  is given by

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$$

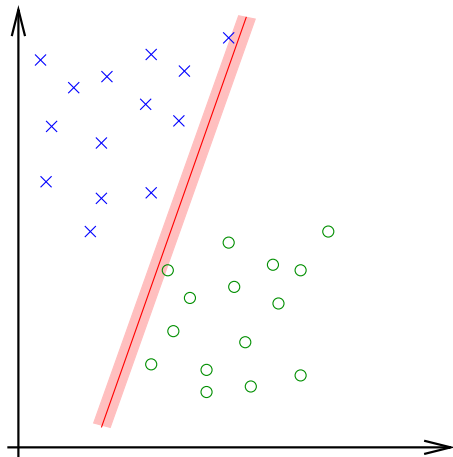
with

$$\text{sign}(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}.$$



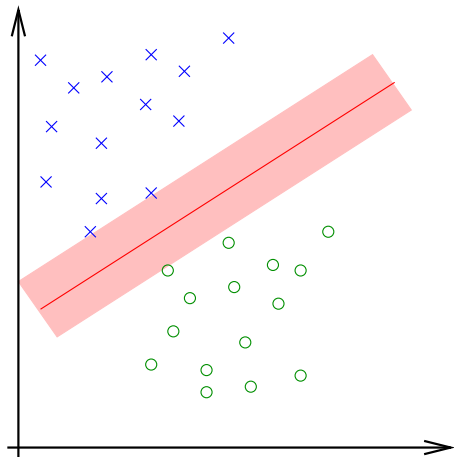
# Maximum margin classifier

- Intuitively, a wide margin around the dividing line makes it more likely that new samples will fall on the right side of the boundary.



# Maximum margin classifier

- Intuitively, a wide margin around the dividing line makes it more likely that new samples will fall on the right side of the boundary.
- Actual rigorous motivation comes from Statistical Learning Theory <sup>1</sup>
- Objective:  
Find a hyperplane that separates both classes with the maximum margin.



<sup>1</sup>V. Vapnik - "Statistical Learning Theory", 1995

# Linear classifier with margin

We add two more hyperplanes that are parallel to the original hyperplane and require that no training points must lie between those hyperplanes.

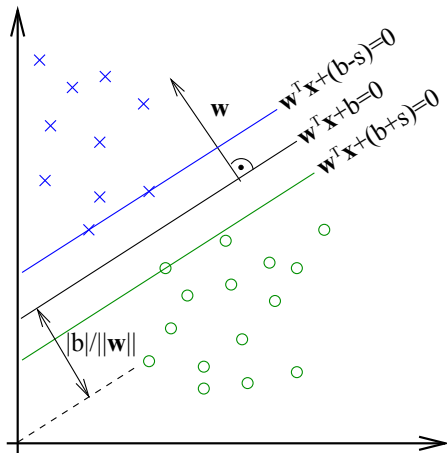
Thus we now require

$$\mathbf{w}^T \mathbf{x} + (b - s) > 0$$

for all  $\mathbf{x}$  from class blue and

$$\mathbf{w}^T \mathbf{x} + (b + s) < 0$$

for all  $\mathbf{x}$  from class green.



# Size of the margin

Signed distance from the origin to the hyperplane is given by

$$d = -\frac{b}{\|w\|}.$$

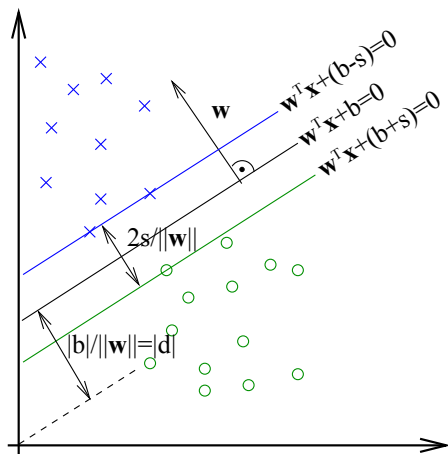
Thus we have

$$d_{blue} = -\frac{b-s}{\|w\|}$$

$$d_{green} = -\frac{b+s}{\|w\|}$$

and the margin is

$$m = d_{blue} - d_{green} = \frac{2s}{\|w\|}.$$



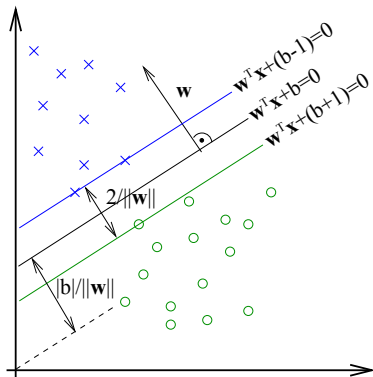
# Redundancy of parameter $s$

The size of the margin,

$$m = \frac{2s}{\|w\|}$$

only depends on the ratio, so w.l.o.g. we can set  $s = 1$  and get

$$m = \frac{2}{\|w\|}$$



# Set of constraints

Let  $\mathbf{x}_i$  be the  $i$ th sample, and  $y_i \in \{-1, 1\}$  the class assigned to  $\mathbf{x}_i$ .

The constraints

$$\mathbf{w}^T \mathbf{x}_i + b \geq +1 \quad \text{for } y_i = +1,$$

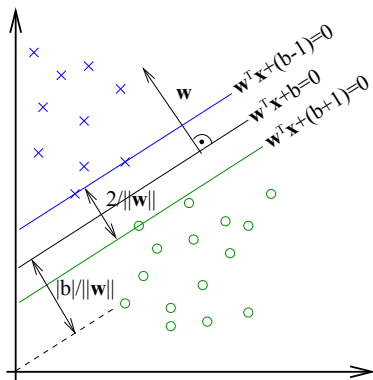
$$\mathbf{w}^T \mathbf{x}_i + b \leq -1 \quad \text{for } y_i = -1$$

can be condensed into

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \text{for all } i.$$

If these constraints are fulfilled the margin is

$$m = \frac{2}{\|\mathbf{w}\|} = \frac{2}{\sqrt{\mathbf{w}^T \mathbf{w}}}.$$



# SVM's Optimization problem

Let  $\mathbf{x}_i$  be the  $i$ th data point,  $i = 1, \dots, N$ , and  $y_i \in \{-1, 1\}$  the class assigned to  $\mathbf{x}_i$ .

To find the separating hyperplane with the maximum margin we need to find  $\{\mathbf{w}, b\}$  that

$$\begin{aligned} \text{minimize} \quad & f_0(\mathbf{w}, b) = \frac{1}{2} \mathbf{w}^T \mathbf{w} \\ \text{subject to} \quad & f_i(\mathbf{w}, b) = y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0 \quad \text{for } i = 1, \dots, N. \end{aligned}$$

This is a **constrained convex optimization problem** (more specifically, quadratic programming problem).

---

We go from  $\|\mathbf{w}\|$  to  $\|\mathbf{w}\|^2 = \mathbf{w}^T \mathbf{w}$  as square root is a monotonic function that doesn't change the location of the optimum.

# Optimization with inequality constraints

## Constrained optimization problem

Given  $f_0 : \mathbb{R}^d \rightarrow \mathbb{R}$  and  $f_i : \mathbb{R}^d \rightarrow \mathbb{R}$ ,

$$\begin{array}{ll}\text{minimize}_{\boldsymbol{\theta}} & f_0(\boldsymbol{\theta}) \\ \text{subject to} & f_i(\boldsymbol{\theta}) \leq 0 \quad \text{for } i = 1, \dots, M.\end{array}$$

## Feasibility

A point  $\boldsymbol{\theta} \in \mathbb{R}^d$  is called **feasible** if and only if it satisfies the constraints of the optimization problem, i.e.  $f_i(\boldsymbol{\theta}) \leq 0$  for all  $i \in \{1, \dots, M\}$ .

## Minimum and minimizer

We call the optimal value the **minimum**  $p^*$ , and the point where the minimum is obtained the **minimizer**  $\boldsymbol{\theta}^*$ . Thus  $p^* = f_0(\boldsymbol{\theta}^*)$ .

# Lagrangian

$$\begin{array}{ll}\text{minimize}_{\boldsymbol{\theta}} & f_0(\boldsymbol{\theta}) \\ \text{subject to} & f_i(\boldsymbol{\theta}) \leq 0 \quad \text{for } i = 1, \dots, M\end{array}$$

## Definition (Lagrangian)

We define the **Lagrangian**  $L : \mathbb{R}^d \times \mathbb{R}^M \rightarrow \mathbb{R}$  associated with the above problem as

$$L(\boldsymbol{\theta}, \boldsymbol{\alpha}) = f_0(\boldsymbol{\theta}) + \sum_{i=1}^M \alpha_i f_i(\boldsymbol{\theta}).$$

We refer to  $\alpha_i \geq 0$  as the **Lagrange multiplier** associated with the inequality constraint  $f_i(\boldsymbol{\theta}) \leq 0$ .

# Lagrange dual function

## Definition (Lagrange dual function)

The **Lagrange dual function**  $g : \mathbb{R}^M \rightarrow \mathbb{R}$  maps  $\alpha$  to the minimum of the Lagrangian over  $\theta$  (possibly  $-\infty$  for some values of  $\alpha$ ),

$$g(\alpha) = \min_{\theta \in \mathbb{R}^d} L(\theta, \alpha) = \min_{\theta \in \mathbb{R}^d} \left( f_0(\theta) + \sum_{i=1}^M \alpha_i f_i(\theta) \right).$$

It is concave in  $\alpha$  since it is the point-wise minimum of a family of affine functions of  $\alpha$ .

# Interpretation of the Lagrangian

$$\begin{array}{ll}\text{minimize}_{\boldsymbol{\theta}} & f_0(\boldsymbol{\theta}) \\ \text{subject to} & f_i(\boldsymbol{\theta}) \leq 0, \quad i = 1, \dots, M\end{array}$$

$$L(\boldsymbol{\theta}, \boldsymbol{\alpha}) = f_0(\boldsymbol{\theta}) + \sum_{i=1}^M \alpha_i f_i(\boldsymbol{\theta})$$

For every choice of  $\boldsymbol{\alpha}$ , the corresponding *unconstrained*  $g(\boldsymbol{\alpha}) = \min_{\boldsymbol{\theta} \in \mathbb{R}^d} L(\boldsymbol{\theta}, \boldsymbol{\alpha})$  is a **lower bound** on the optimal value of the constrained problem:

$$\min_{\substack{\boldsymbol{\theta} \in \mathbb{R}^d \\ f_i(\boldsymbol{\theta}) \leq 0}} f_0(\boldsymbol{\theta}) = f_0(\boldsymbol{\theta}^*) \geq f_0(\boldsymbol{\theta}^*) + \sum_{i=1}^M \underbrace{\alpha_i}_{\geq 0} \underbrace{f_i(\boldsymbol{\theta}^*)}_{\leq 0} = L(\boldsymbol{\theta}^*, \boldsymbol{\alpha}) \geq \underbrace{\min_{\boldsymbol{\theta} \in \mathbb{R}^d} L(\boldsymbol{\theta}, \boldsymbol{\alpha})}_{g(\boldsymbol{\alpha})}$$

Hence,  $\forall \boldsymbol{\alpha} \quad f_0(\boldsymbol{\theta}^*) \geq g(\boldsymbol{\alpha})$ .

# Lagrange dual problem

For each  $\alpha \geq 0$  the Lagrange dual function  $g(\alpha)$  gives us a lower bound on the optimal value  $p^*$  of the original optimization problem.

What is the best (highest) lower bound?

## Lagrange dual problem

$$\begin{aligned} & \text{maximize}_{\alpha} && g(\alpha) \\ & \text{subject to} && \alpha_i \geq 0, \quad i = 1, \dots, m \end{aligned}$$

The maximum  $d^*$  of the Lagrange dual problem is the best lower bound on  $p^*$  that we can achieve by using the Lagrangian.

*Note: since we are maximizing  $g$  we are not interested in dual multipliers  $\alpha$  such that  $g(\alpha) = -\infty$ , so the condition  $g(\alpha) \neq -\infty$  is usually added as an additional constraint to the dual problem  $\rightarrow$  we call  $\alpha$  feasible if and only if all  $\alpha_i \geq 0$  and  $L(\theta, \alpha)$  is bounded from below for  $\theta \in \mathbb{R}^d$ .*

# Duality

## Weak duality (always)

Since for all  $\alpha \geq \mathbf{0}$  it holds that  $g(\alpha) \leq p^*$  we have **weak duality**,

$$d^* \leq p^* .$$

The difference  $p^* - d^* \geq 0$  between the solution of the original and the dual problem is called the **duality gap**.

## Strong duality (under certain conditions)

Under certain conditions we have

$$d^* = p^* ,$$

i.e. the maximum to the Lagrange dual problem is the minimum of the original (primal) constrained optimization problem (i.e.,  $f_0(\theta^*) = g(\alpha^*)$ ).

# Complementary Slackness

Given strong duality and that we have found an optimal solution  $\theta^*$  to the primal problem, as well as an optimal solution  $\alpha^*$  to the dual problem. Then we have

$$\begin{aligned} f_0(\theta^*) &= g(\alpha^*) \\ &= f_0(\theta^*) + \sum_{i=1}^M \alpha_i^* f_i(\theta^*) \quad \rightarrow \quad \sum_{i=1}^M \alpha_i^* f_i(\theta^*) = 0 \end{aligned}$$

Now, as  $\alpha_i^* \geq 0$  and  $f_i(\theta^*) \leq 0$  it follows that  $\alpha_i^* f_i(\theta^*) = 0$  for all  $i \geq 1$ .

This condition is known as **complementary slackness**.

# SVM's Primal problem

We apply a recipe for solving the constrained optimization problem.

1. Calculate the Lagrangian

$$L(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_{i=1}^N \alpha_i [y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1].$$

2. Minimize  $L(\mathbf{w}, b, \boldsymbol{\alpha})$  w.r.t.  $\mathbf{w}$  and  $b$ .

$$\nabla_{\mathbf{w}} L(\mathbf{w}, b, \boldsymbol{\alpha}) = \mathbf{w} - \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i \stackrel{!}{=} 0$$

$$\frac{\partial L}{\partial b} = \sum_{i=1}^N \alpha_i y_i \stackrel{!}{=} 0$$

Thus the weights are a linear combination of the training samples,

$$\mathbf{w} = \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i.$$

# SVM's Dual problem

Substituting both relations back into  $L(\mathbf{w}, b, \boldsymbol{\alpha})$  gives the Lagrange dual function  $g(\boldsymbol{\alpha})$ .

Thus we have reformulated our original problem as

$$\begin{aligned} \text{maximize} \quad & g(\boldsymbol{\alpha}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{subject to} \quad & \sum_{i=1}^N \alpha_i y_i = 0 \\ & \alpha_i \geq 0, \quad \text{for } i = 1, \dots, N. \end{aligned}$$

3. Solve this problem.

# Solving the dual problem

We can rewrite the dual function  $g(\alpha)$  in vector form

$$g(\alpha) = \frac{1}{2} \alpha^T Q \alpha + \alpha^T \mathbf{1}_N$$

where  $Q$  is a symmetric negative (semi-)definite matrix, and the constraints on  $\alpha$  are linear.

This is an instance of a [quadratic programming](#) problem. There exist efficient algorithms for its solution, such as [Sequential minimal optimization](#) (SMO) <sup>2</sup>.

A number of implementations, such as LIBSVM <sup>3</sup> are available and are widely used in practice.

---

<sup>2</sup><http://cs229.stanford.edu/materials/smo.pdf>

<sup>3</sup>C.-C. Chang and C.-J. Lin. *LIBSVM: a library for support vector machines*, 2011

## Recovering $\mathbf{w}$ and $b$ from the dual solution $\alpha^*$

Having obtained the optimal  $\alpha^*$  using our favorite QP solver, we can compute the parameters defining the separating hyperplane.

Recall, that from minimizing the Lagrangian, the corresponding optimal weights  $\mathbf{w}$  are a linear combination of the training samples,

$$\mathbf{w} = \sum_{i=1}^N \alpha_i^* y_i \mathbf{x}_i$$

From the complementary slackness condition  $\alpha_i^* f_i(\boldsymbol{\theta}^*) = 0$  we can recover the bias:

Take any  $\mathbf{x}_i$  for which  $\alpha_i \neq 0$ . Due to complementary slackness, the corresponding constraint  $f_i(\mathbf{w}, b)$  must be zero. Thus, we have

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1.$$

Solving this for  $b$  yields the bias

$$b = 1/y_i - \mathbf{w}^T \mathbf{x}_i = y_i - \mathbf{w}^T \mathbf{x}_i$$

---

We can also average the  $b$  over all support vectors to get a more stable solution.

# Support vectors

From complimentary slackness

$$\alpha_i^* f_i(\boldsymbol{\theta}^*) = 0.$$

we have

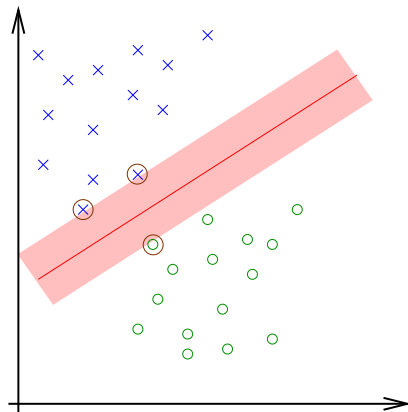
$$\alpha_i [y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) - 1] = 0 \quad \text{for all } i.$$

Hence a training sample  $\boldsymbol{x}_i$  can only contribute to the weight vector ( $\alpha_i \neq 0$ ) if it lies **on the margin**, that is

$$y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) = 1.$$

A training sample  $\boldsymbol{x}_i$  with  $\alpha_i \neq 0$  is called a **support vector**. Let's collect the indices of all support vectors in the set  $\mathcal{S} = \{i \mid \alpha_i \neq 0\}$ .

$$\boldsymbol{w} = \sum_{i=1}^N \alpha_i y_i \boldsymbol{x}_i$$



# Classifying

The class of  $\mathbf{x}$  is given by

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b).$$

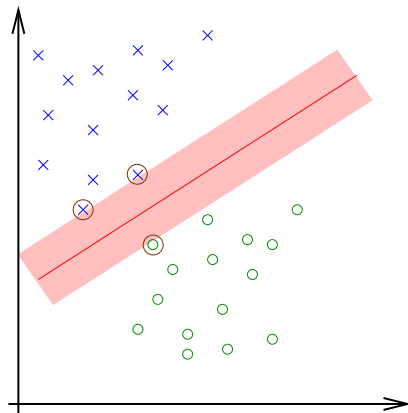
Substituting

$$\mathbf{w} = \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i = \sum_{i \in \mathcal{S}} \alpha_i y_i \mathbf{x}_i$$

gives

$$h(\mathbf{x}) = \text{sign} \left( \sum_{i \in \mathcal{S}} \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b \right).$$

Since the solution is **sparse** (most  $\alpha_i$ s are zero) we only need to remember the few training samples in  $\mathcal{S}$ , i.e., the  $\mathbf{x}_i$  with  $\alpha_i \neq 0$ .



## Section 2

### Soft Margin Support Vector Machines

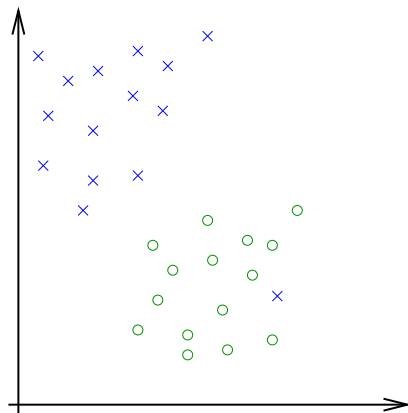
# Dealing with noisy data

What if the data is not linearly separable due to some noise?

With our current version of SVM, a single outlier makes the constraint unsatisfiable.

The corresponding Lagrange multiplier  $\alpha_i$  would go to infinity and destroy the solution.

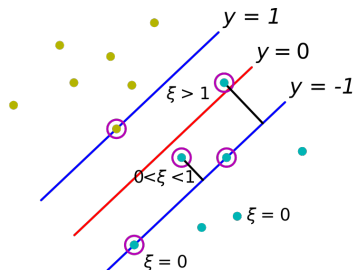
How to make our model more robust?



# Slack variables

Idea: Relax the constraints as much as necessary but punish the relaxation of a constraint.

We introduce a **slack variable**  $\xi_i \geq 0$  for every training sample  $x_i$  that gives the distance of how far the margin is violated by this training sample in units of  $\|w\|$ .



# Slack variables

Idea: Relax the constraints as much as necessary but punish the relaxation of a constraint.

We introduce a **slack variable**  $\xi_i \geq 0$  for every training sample  $x_i$  that gives the distance of how far the margin is violated by this training sample in units of  $\|w\|$ .

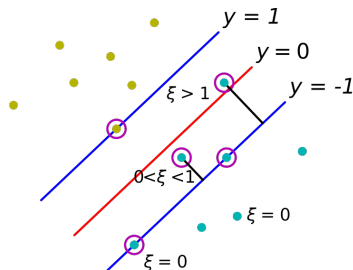
Hence our relaxed constraints are

$$w^T x_i + b \geq +1 - \xi_i \quad \text{for } y_i = +1,$$

$$w^T x_i + b \leq -1 + \xi_i \quad \text{for } y_i = -1.$$

Again, they can be condensed into

$$y_i(w^T x_i + b) \geq 1 - \xi_i \quad \text{for all } i.$$



# Slack variables

Idea: Relax the constraints as much as necessary but punish the relaxation of a constraint.

We introduce a **slack variable**  $\xi_i \geq 0$  for every training sample  $\mathbf{x}_i$  that gives the distance of how far the margin is violated by this training sample in units of  $\|\mathbf{w}\|$ .

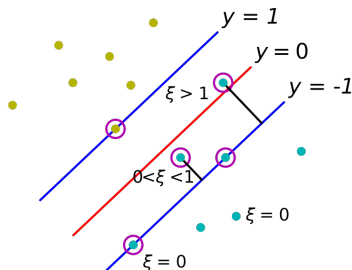
Hence our relaxed constraints are

$$\mathbf{w}^T \mathbf{x}_i + b \geq +1 - \xi_i \quad \text{for } y_i = +1,$$

$$\mathbf{w}^T \mathbf{x}_i + b \leq -1 + \xi_i \quad \text{for } y_i = -1.$$

Again, they can be condensed into

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \quad \text{for all } i.$$



The new cost function is,

$$f_0(\mathbf{w}, b, \boldsymbol{\xi}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \xi_i.$$

The factor  $C > 0$  determines how heavy a violation is punished.

$C \rightarrow \infty$  recovers hard-margin SVM.

# Optimization problem with slack variables

Let  $\mathbf{x}_i$  be the  $i$ th data point,  $i = 1, \dots, N$ , and  $y_i \in \{-1, 1\}$  the class assigned to  $\mathbf{x}_i$ . Let  $C > 0$  be a constant.

To find the hyperplane that separates most of the data with maximum margin we

$$\begin{aligned} \text{minimize} \quad & f_0(\mathbf{w}, b, \boldsymbol{\xi}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \xi_i \\ \text{subject to} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i \geq 0 & i = 1, \dots, N, \\ & \xi_i \geq 0 & i = 1, \dots, N. \end{aligned}$$

Here we used the 1-norm for the penalty term  $\sum_i \xi_i$ . Another choice is to use the 2-norm penalty,  $\sum_i \xi_i^2$ .

The penalty that performs better in practice will depend on the data and the type of noise that has influenced it.

# Lagrangian with slack variables

## 1. Calculate the Lagrangian

$$L(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \xi_i \\ - \sum_{i=1}^N \alpha_i [y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i] - \sum_{i=1}^N \mu_i \xi_i.$$

## 2. Minimize $L(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu})$ w.r.t. $\mathbf{w}$ , $b$ and $\boldsymbol{\xi}$ .

$$\nabla_{\mathbf{w}} L(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \mathbf{w} - \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i \stackrel{!}{=} 0; \quad \frac{\partial L}{\partial b} = \sum_{i=1}^N \alpha_i y_i \stackrel{!}{=} 0, \\ \frac{\partial L}{\partial \xi_i} = C - \alpha_i - \mu_i \stackrel{!}{=} 0 \quad \text{for } i = 1, \dots, N$$

From  $\alpha_i = C - \mu_i$  and dual feasibility  $\mu_i \geq 0$ ,  $\alpha_i \geq 0$  we get

$$0 \leq \alpha_i \leq C.$$

# Dual problem with slack variables

This leads to the dual problem:

$$\begin{aligned} \text{maximize} \quad & g(\boldsymbol{\alpha}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{subject to} \quad & \sum_{i=1}^N \alpha_i y_i = 0 \\ & 0 \leq \alpha_i \leq C \quad i = 1, \dots, N. \end{aligned}$$

This is nearly the same dual problem as for the case without slack variables.

Only the constraint  $\alpha_i \leq C$  is new. It ensures that  $\alpha_i$  is bounded and cannot go to infinity.

# Classifying a new point with the soft margin SVM

Denote the set of indices of support vectors as  $\mathcal{S}$  (i.e., the indices for the  $\mathbf{x}_i$  for which  $0 < \alpha_i \leq C$ ). Thus, we can again write  $\mathbf{w} = \sum_{i \in \mathcal{S}} \alpha_i y_i \mathbf{x}_i$ . Note: If  $0 < \alpha_i < C$  then  $\xi_i = 0$ ; if  $\alpha_i = C$  then  $\xi_i > 0$ .

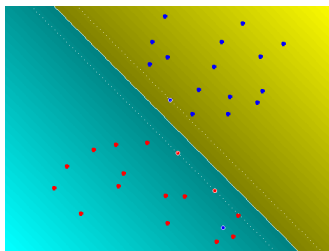
Due to complementary slackness, the support vectors  $\mathbf{x}_i$ ,  $i \in \mathcal{S}$  with  $\xi_i = 0$  must satisfy  $y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 = 0$ . Therefore, like for the regular SVM, the bias can be recovered as

$$b = \frac{1}{y_i} - \mathbf{w}^T \mathbf{x}_i = y_i - \left( \sum_{j \in \mathcal{S}} \alpha_j y_j \mathbf{x}_i^T \mathbf{x}_j \right)$$

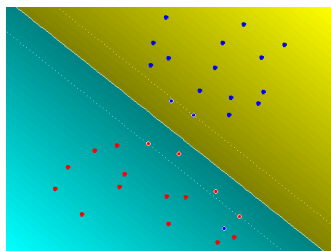
As a result, a new point  $\mathbf{x}$  can be classified as

$$h(\mathbf{x}) = \text{sign} \left( \sum_{j \in \mathcal{S}} \alpha_j y_j \mathbf{x}_j^T \mathbf{x} + b \right)$$

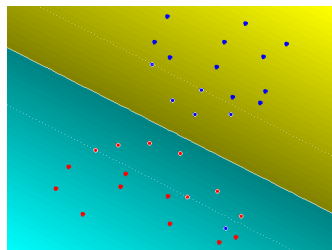
# Influence of the penalty $C$



$C = 100$



$C = 10$



$C = 1$

# Hinge loss formulation

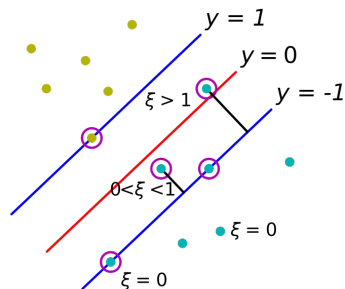
We can have another look at our **constrained** optimization problem.

$$\begin{aligned} \text{minimize} \quad & f_0(\mathbf{w}, b, \boldsymbol{\xi}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \xi_i \\ \text{subject to} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i \geq 0, \quad i = 1, \dots, N, \\ & \xi_i \geq 0, \quad i = 1, \dots, N. \end{aligned}$$

Clearly, for the optimal solution the slack variables are

$$\xi_i = \begin{cases} 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b), & \text{if } y_i(\mathbf{w}^T \mathbf{x}_i + b) < 1 \\ 0 & \text{else} \end{cases}$$

since we are minimizing over  $\boldsymbol{\xi}$ .



# Hinge loss formulation

Thus, we can rewrite the objective function as an **unconstrained** optimization problem known as the **hinge loss** formulation

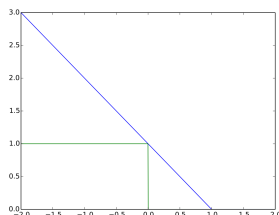
$$\min_{\mathbf{w}, b} \quad \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \max\{0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)\}$$

The **hinge loss** function

$E_{\text{hinge}}(z) = \max\{0, 1 - z\}$  penalizes the points that lie within the margin.

The name comes from the shape from the function, as can be seen in the figure to the right.

We can optimize this hinge loss objective directly, using standard gradient-based methods.



Hinge loss (**blue**) can be viewed as an approximation to zero-one loss (**green**).

## Section 3

### Kernels

# Feature space

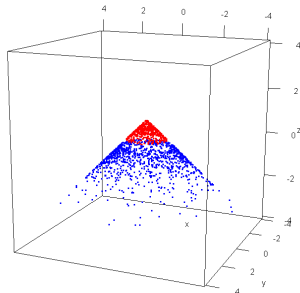
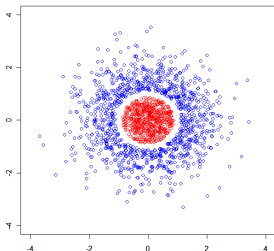
So far we can only construct linear classifiers.

Before, we used **basis functions**  $\phi(\cdot)$  to make the models nonlinear

$$\phi : \mathbb{R}^D \rightarrow \mathbb{R}^M \quad \mathbf{x}_i \mapsto \phi(\mathbf{x}_i)$$

For example, with the following mapping the data becomes linearly separable

$$\phi(x, y) = \begin{pmatrix} x \\ y \\ -\sqrt{x^2 + y^2} \end{pmatrix}$$



# Kernel trick

In the dual formulation of SVM, the samples  $\mathbf{x}_i$  only enter the dual objective as **inner products**  $\mathbf{x}_i^T \mathbf{x}_j$

$$g(\boldsymbol{\alpha}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \mathbf{x}_i^T \mathbf{x}_j ,$$

For basis functions this means that

$$g(\boldsymbol{\alpha}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

# Kernel trick

We can define a **kernel function**  $k : \mathbb{R}^D \times \mathbb{R}^D \rightarrow \mathbb{R}$

$$k(\mathbf{x}_i, \mathbf{x}_j) := \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

and rewrite the dual as

$$g(\boldsymbol{\alpha}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j k(\mathbf{x}_i, \mathbf{x}_j)$$

This operation is referred to as **the kernel trick**.

It can be used not only for SVM. Kernel trick can be used in any model that can be formulated such that it only depends on the inner products  $\mathbf{x}_i^T \mathbf{x}_j$ . (e.g. linear regression, k-nearest neighbors)

# Classifying a new point with kernelized SVM

Denote the set of indices of support vectors as  $\mathcal{S}$  (i.e., the indices for the points  $\mathbf{x}_i$  for which holds  $0 < \alpha_i \leq C$ ). Note: If  $0 < \alpha_i < C$  then  $\xi_i = 0$ ; if  $\alpha_i = C$  then  $\xi_i > 0$ .

From the complementary slackness condition, the support vectors  $\mathbf{x}_i$ ,  $i \in \mathcal{S}$  with  $\xi_i = 0$  must satisfy  $y_i \left( \sum_{j \in \mathcal{S}} \alpha_j y_j k(\mathbf{x}_i, \mathbf{x}_j) + b \right) = 1$ .

Then, like for the regular SVM, the bias can be recovered as

$$b = y_i - \left( \sum_{j \in \mathcal{S}} \alpha_j y_j k(\mathbf{x}_i, \mathbf{x}_j) \right)$$

Thus, a new point  $\mathbf{x}$  can be classified as

$$h(\mathbf{x}) = \text{sign} \left( \sum_{j \in \mathcal{S}} \alpha_j y_j k(\mathbf{x}_j, \mathbf{x}) + b \right)$$

# Advantages of the kernel trick

The kernel represent an inner product in the **feature space** spanned by  $\phi$ . Like before, this makes our models non-linear w.r.t. the data space.

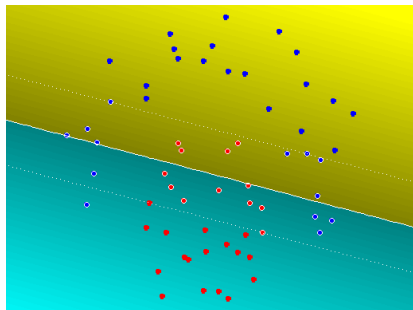
What's the point of using kernels if we can simply use the basis functions?

- Some kernels are equivalent to using infinite-dimensional basis functions. While computing these feature transformations would be impossible, directly evaluating the kernels is often easy.
- Kernels can be used to encode similarity between arbitrary non-numerical data, from strings to graphs.

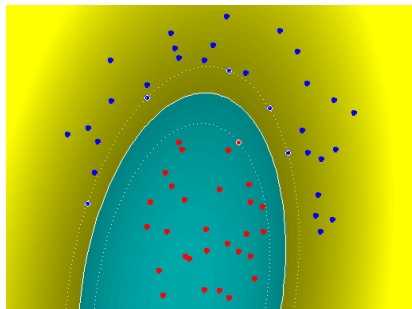
For example, we could define

$$k(\text{lemon}, \text{orange}) = 10 \quad \text{and} \quad k(\text{apple}, \text{orange}) = -5$$

# SVM using kernels example



Linear kernel (no kernel)  
 $\mathbf{a}^T \mathbf{b}$



2nd order polynomial kernel  
 $(\mathbf{a}^T \mathbf{b})^2$

# What makes a valid kernel?

A kernel is **valid** if it corresponds to an inner product in some feature space. An equivalent formulation is given by Mercer's theorem.

## Mercer's theorem

A kernel is valid if it gives rise to a **symmetric, positive semidefinite** kernel matrix  $\mathbf{K}$  for any input data  $\mathbf{X}$ .

**Kernel matrix** (also known as **Gram matrix**)  $\mathbf{K} \in \mathbb{R}^{N \times N}$  is defined as

$$\mathbf{K} = \begin{pmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & k(\mathbf{x}_1, \mathbf{x}_2) & \cdots & k(\mathbf{x}_1, \mathbf{x}_N) \\ k(\mathbf{x}_2, \mathbf{x}_1) & k(\mathbf{x}_2, \mathbf{x}_2) & \cdots & k(\mathbf{x}_2, \mathbf{x}_N) \\ \vdots & \vdots & \ddots & \vdots \\ k(\mathbf{x}_N, \mathbf{x}_1) & k(\mathbf{x}_N, \mathbf{x}_2) & \cdots & k(\mathbf{x}_N, \mathbf{x}_N) \end{pmatrix}$$

What happens if we use a non-valid kernel?

Our optimization problem might become non-convex, so we may not get a globally optimal solution.

# Kernel preserving operations

Let  $k_1 : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$  and  $k_2 : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$  be kernels, with  $\mathcal{X} \subseteq \mathbb{R}^N$ . Then the following functions  $k$  are kernels as well:

- $k(\mathbf{x}_1, \mathbf{x}_2) = k_1(\mathbf{x}_1, \mathbf{x}_2) + k_2(\mathbf{x}_1, \mathbf{x}_2)$
- $k(\mathbf{x}_1, \mathbf{x}_2) = c \cdot k_1(\mathbf{x}_1, \mathbf{x}_2)$ , with  $c > 0$
- $k(\mathbf{x}_1, \mathbf{x}_2) = k_1(\mathbf{x}_1, \mathbf{x}_2) \cdot k_2(\mathbf{x}_1, \mathbf{x}_2)$
- $k(\mathbf{x}_1, \mathbf{x}_2) = k_3(\phi(\mathbf{x}_1), \phi(\mathbf{x}_2))$ , with the kernel  $k_3$  on  $\mathcal{X}' \subseteq \mathbb{R}^M$  and  $\phi : \mathcal{X} \rightarrow \mathcal{X}'$
- $k(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{x}_1^T \mathbf{A} \mathbf{x}_2$ , with  $\mathbf{A} \in \mathbb{R}^N \times \mathbb{R}^N$  symmetric and positive semidefinite

# Examples of kernels

- Polynomial:

$$k(\mathbf{a}, \mathbf{b}) = (\mathbf{a}^T \mathbf{b})^p \quad \text{or} \quad (\mathbf{a}^T \mathbf{b} + 1)^p$$

- Gaussian kernel:

$$k(\mathbf{a}, \mathbf{b}) = \exp \left( -\frac{\|\mathbf{a} - \mathbf{b}\|^2}{2\sigma^2} \right)$$

- Sigmoid:

$$k(\mathbf{a}, \mathbf{b}) = \tanh(\kappa \mathbf{a}^T \mathbf{b} - \delta) \quad \text{for } \kappa, \delta > 0$$

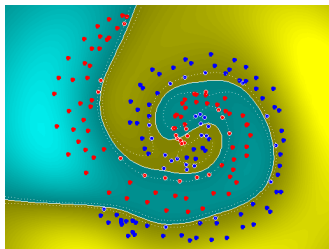
In fact, the sigmoid kernel **is not** PSD, but still works well in practice.

Some kernels introduce additional hyperparameters, that affect the behavior of the algorithm.

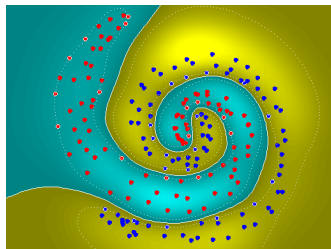
---

Note, that the sigmoid kernel is different from the sigmoid function from *Linear Classification*.

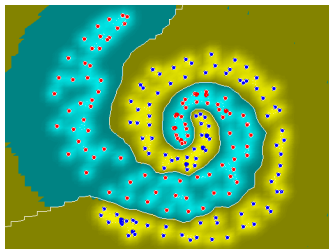
# Gaussian kernel ( $C=1000$ )



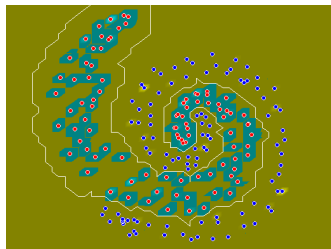
$\sigma = 0.5$



$\sigma = 0.25$



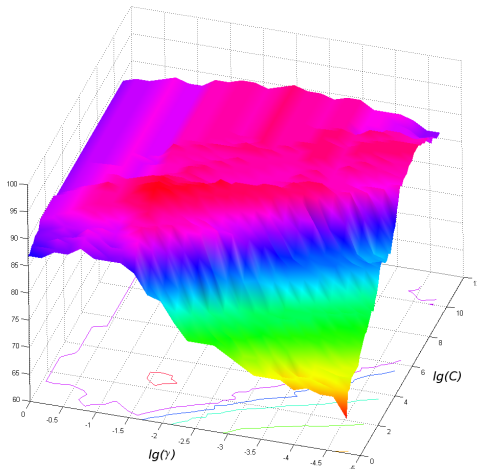
$\sigma = 0.05$



$\sigma = 0.005$

# How to choose the hyperparameters?

The best setting of the penalty parameter  $C$  and the kernel hyperparameters (e.g.,  $\gamma$  or  $\sigma$ ) can be determined by performing **cross validation** with **random search** over the parameter space.



# Multiple classes

The standard SVM model cannot handle multiclass data.

Two approaches to address this issue are:

- One-vs-rest:** Train  $C$  SVM models for  $C$  classes, where each SVM is being trained for classification of one class against all the remaining ones. The winner is then the class, where the distance from the hyperplane is maximal.
- One-vs-one:** Train  $\binom{C}{2}$  classifiers (all possible pairings) and evaluate all. The winner is the class with the majority vote (votes are weighted according to the distance from the margin).

# Summary

- Support Vector Machine is a linear binary classifier that, motivated by learning theory, maximizes the margin between the two classes.
- The SVM objective is convex, so a globally optimal solution can be obtained.
- The dual formulation is a quadratic programming problem, that can be solved efficiently, e.g. using standard QP libraries.
- Soft-margin SVM with slack variables can deal with noisy data. Smaller values for the penalty  $C$  lead to a larger margin at the cost of misclassifying more samples.
- Linear soft-margin SVM (= hinge loss formulation) can be solved directly using gradient descent.
- We can obtain a nonlinear decision boundary by moving to an implicit high-dimensional feature space by using the kernel trick. This only works in the dual formulation.

# Reading material

## Reading material

- Bishop: chapters 7.1.0, 7.1.1, 7.1.2

## Acknowledgements

- Slides are based on an older version by S. Urban and modifications by A. Bojevski